Unit 1

1a. What is a System?

A **system** is a set of **interrelated components** that work together to achieve a common goal. It takes **inputs**, processes them, and produces **outputs**.

Example:

- A payroll system takes employee data (input), calculates salaries (process), and generates payslips (output).

2. Types of Systems

1. **Physical Systems** – Tangible systems you can touch.
    - Example: Computer, ATM, Manufacturing machine
2. **Abstract Systems** – Intangible systems, ideas, or methods.
    - Example: Business rules, mathematical models
3. **Open Systems** – Interact with the environment and receive feedback.
    - Example: Online shopping system, bank system
4. **Closed Systems** – No interaction with the environment, self-contained.
    - Example: Automatic control system
5. **Man-Made Systems** – Designed and created by humans.
    - Example: Library management system, Hospital management system
6. **Natural Systems** – Occur naturally in the environment.
    - Example: Solar system, ecosystem

a2. Advantages of Using Systems Analysis and Design Techniques in Computerized Information Systems

1. **Better Understanding of Requirements** – Helps to identify what the system should do.
2. **Efficient Design** – Ensures systems are structured and organized.
3. **Error Reduction** – Detects problems early in analysis and design phases.
4. **Cost-Effective** – Reduces unnecessary rework and waste of resources.
5. **Improved Communication** – Between users, managers, and developers.
6. **Enhanced Decision Making** – Provides accurate data and system insights.
7. **Documentation** – Helps in future maintenance and upgrades.
8. **User Satisfaction** – Ensures the system meets user needs and expectations.

2a.List and briefly define the seven phases of the systems development life cycle (SDLC).

1. Seven Phases of SDLC

The **Systems Development Life Cycle (SDLC)** is a step-by-step process used to develop information systems. The seven phases are:

1. **Planning** – Identify the need for a new system, define goals, and prepare a project plan.
2. **System Analysis** – Study the current system, gather requirements, and identify problems.
3. **System Design** – Design the new system's architecture, inputs, outputs, and processes.
4. **Development / Implementation** – Convert designs into actual software or system components.
5. **Testing** – Check the system for errors, bugs, and ensure it meets requirements.
6. **Deployment / Installation** – Install the system in the real environment and make it operational.
7. **Maintenance** – Update, fix, and improve the system over time to keep it efficient.

2.bWhat are CASE tools used for?

Definition of CASE Tools

CASE (Computer-Aided Software Engineering) tools are software applications that help **system analysts, designers, and developers** to **plan, analyze, design, and implement information systems** efficiently. They automate or support various tasks in the **software development life cycle (SDLC)**.

Uses of CASE Tools

1. **System Modeling**
    - Helps in creating **visual models** like Data Flow Diagrams (DFDs), Entity-Relationship Diagrams (ERDs), and UML diagrams.

- Makes it easier to understand complex systems.

2. **Documentation**
    - Automatically generates system documentation.
    - Reduces manual effort and ensures consistency.
3. **Code Generation**
    - Some CASE tools can **automatically generate code** from system models.
    - Speeds up development and reduces errors.
4. **Design Validation**
    - Helps in checking the **correctness and completeness** of system design before implementation.
5. **Project Management**
    - Tracks project progress, manages resources, and schedules tasks.
    - Provides reports on timelines, costs, and workloads.
6. **Maintenance Support**
    - Helps maintain and update systems by keeping track of design, code, and documentation.
7. **Error Reduction**
    - Minimizes errors in design and coding by using automated checking tools.

Q3A.Differentiate between upper- and lower-case tools

-

Point	Upper-CASE Tools	Lower-CASE Tools
1	Used in **early stages** of SDLC	Used in **later stages** of SDLC
2	Helps in **system analysis & design**	Helps in **coding & testing**
3	Focus on **planning and modeling**	Focus on **building and implementing**
4	Creates **diagrams and models**	Generates **source code**
5	Produces **documentation**	Helps in **debugging & maintenance**
6	Used by **system analysts & designers**	Used by **programmers & developers**
7	Mostly **semi-automated**	Mostly **automated**
8	Examples: **ERwin, Rational Rose, MS Visio**	Examples: **Oracle Designer, PowerBuilder, Visual Paradigm**

9	Helps to **understand system clearly**	Helps to **develop system faster**
10	Called **“Think & Plan” tools**	Called **“Build & Implement” tools**

q3b.Define what is meant by the agile approach. What are the stages in agile development?

Definition of Agile Approach

The **Agile approach** is a **flexible and iterative method** of software development.

- It focuses on **delivering small parts of the system quickly**.
- It encourages **customer collaboration**, **frequent feedback**, and **adaptation to changes**.
- Instead of completing the whole system at once, Agile develops the system in **small iterations called sprints**.

✅ **Key idea:** “**Change is welcome, work in small steps, deliver quickly**.”

Stages in Agile Development

Agile development follows a **cyclic iterative process**. The main stages are:

Stage	Description
1. **Planning**	Gather requirements from the customer and plan the sprint.
2. **Design**	Create a simple design for the current sprint or feature.

3. **Development (Coding)**	Write code and implement the planned features.
4. **Testing**	Test the developed features for errors and correctness.
5. **Review / Feedback**	Show the working software to the customer and collect feedback.
6. **Deployment**	Deploy the tested and approved features to the live environment.
7. **Iteration / Next Sprint**	Repeat the cycle for the next set of requirements until the full system is complete.

4a. Purpose of the Requirements-Gathering Phase in SDLC

The **requirements-gathering phase** (also called **requirements analysis**) is the **first important step in the System Development Life Cycle (SDLC)**.

Purpose:

1. To **understand what the user wants** from the system.
2. To **identify all the functional and non-functional requirements** (what the system should do and how it should perform).
3. To **collect accurate information** from stakeholders (users, managers, etc.).
4. To **avoid misunderstandings** between users and developers.
5. To **create a foundation for system design**, ensuring the system meets business needs.

✅ **In simple words:**

> "This phase tells the developers exactly **what the system should do** before they start designing or coding."

4b. Organizational Subsystems are Interrelated and Interdependent

Meaning:

- An **organization** is made up of **different subsystems** (like HR, Finance, Production, Sales).
- These subsystems **depend on each other** to function properly.
- **Interrelated** means the subsystems **affect each other**.
- **Interdependent** means the subsystems **cannot work alone**; the success of one subsystem depends on the others.

Example:

- HR needs payroll info from Finance to pay salaries.
- Sales depends on Production to deliver goods.
- If one subsystem fails, the others are affected.

✅ **In simple words:**

> "All parts of an organization **work together** and **rely on each other** to achieve overall goals."

5a.How does SDLC help in minimizing project failures

How SDLC Helps in Minimizing Project Failures

The **System Development Life Cycle (SDLC)** is a **structured approach** for developing information systems. It helps reduce failures in the following ways:

1. **Clear Requirements** – By collecting detailed requirements at the start, it ensures the system meets **user needs**.
2. **Proper Planning** – SDLC defines **phases, tasks, and timelines**, avoiding confusion and delays.

3. **Structured Design** – Helps in creating a **well-thought-out system design**, reducing errors during development.

4. **Early Detection of Errors** – Testing in SDLC phases identifies and corrects problems **before deployment**.

5. **Better Communication** – Involves stakeholders at every stage, so **misunderstandings are minimized**.

6. **Resource Management** – Efficient use of **time, money, and manpower** reduces project risks.

7. **Documentation** – Each phase is documented, making it **easier to maintain and update** the system.

8. **Step-by-Step Approach** – Breaking the project into **phases** ensures nothing is overlooked.

9. **Change Management** – SDLC allows **controlled changes**, preventing project chaos.

10. **Quality Assurance** – Ensures the final system is **reliable, functional, and meets user expectations**.

6a.Define the role of a System Analyst.

1. Role of a System Analyst

A **System Analyst** is the **bridge between users and developers**. Their main role is to understand business needs and design systems accordingly.

Key Roles:

1. **Requirement Gathering** – Collect information from users about what they need.

2. **System Analysis** – Analyze requirements to find the best solution.

3. **System Design** – Help design the system structure and processes.

4. **Communication** – Act as a link between **users** and **developers**.

5. **Problem Solving** – Identify problems in the current system and suggest improvements.

6. **Documentation** – Prepare reports, specifications, and diagrams for developers.

7. **Testing & Validation** – Ensure the system meets user requirements after development.

6b.Describe the differences between the Waterfall and Agile models in SDLC

2. Differences between Waterfall and Agile Models

Feature	Waterfall Model	Agile Model
Approach	Linear and sequential	Iterative and incremental
Phases	Fixed phases (Requirement → Design → Implementation → Testing → Deployment)	Cycles of small iterations called **sprints**
Flexibility	**Changes are difficult** once a phase is completed	**Changes are welcomed** anytime during development
Customer Involvement	Limited, mostly at start and end	Continuous, customer feedback in every sprint
Delivery	Final product delivered at the end	Partial working product delivered after each iteration
Risk	Higher risk if requirements change	Lower risk, flexible to changes
Testing	Testing happens at the end	Testing happens in every iteration
Example Use	Projects with **well-defined requirements**	Projects where **requirements may change frequently**

7a. Five Criteria for Systems Project Selection

When choosing which system projects to develop, organizations usually consider these **five criteria**:

1. **Strategic Alignment** – Does the project support the organization's **goals and strategy**?
2. **Return on Investment (ROI)** – Will the project provide **financial benefits** compared to its cost?
3. **Risk** – What are the **risks** involved in the project (technical, financial, operational)?

4. **Resource Availability** – Are there enough **people, time, and budget** to complete the project?

5. **Urgency / Priority** – How **important or urgent** is the project for the organization?

✅ **In simple words:**

> "Projects are selected based on **benefits, risks, resources, alignment with goals, and urgency**."

7b. Feasibility in Systems Development

Feasibility study checks whether a project is **possible and practical** before development.

a) Technical Feasibility

- Determines if the organization has the **technology, hardware, software, and technical skills** to build the system.
- ✅ Example: Can the system run on existing computers or do we need new technology?

b) Economic Feasibility

- Determines whether the **financial benefits** of the system are **greater than the cost**.
- ✅ Example: Will the system save money, increase revenue, or reduce costs enough to justify the investment?

c) Operational Feasibility

- Determines if the system **fits the organization's processes and people**.
- Checks whether users will **accept and use the system effectively**.
- ✅ Example: Will employees adapt to the new system, or will it disrupt daily operations?

Here's a **simple and exam-friendly answer** for your questions:

8a. Three Broad Horizontal Levels of Management

Organizations usually have **three levels of management**, each with different responsibilities:

1. **Top-Level Management**
    - Examples: CEO, Managing Director
    - Responsible for **strategic decisions** and long-term planning.
    - Sets **goals, policies, and overall direction** of the organization.
2. **Middle-Level Management**
    - Examples: Department Heads, Managers
    - Responsible for **tactical decisions** and implementing strategies.
    - Coordinates between **top-level and lower-level management**.
3. **Lower-Level (Operational) Management**
    - Examples: Supervisors, Team Leaders
    - Responsible for **day-to-day operations** and supervising staff.
    - Ensures **tasks are completed efficiently and on time**.

✅ **In simple words:**

> "Top-level plans, middle-level coordinates, and lower-level executes daily tasks."

8b. Importance of a Feasibility Study in Software Development

A **feasibility study** is conducted **before building a system** to check if the project is **practical and worthwhile**.

Importance:

1. **Reduces Risk** – Identifies potential problems early to avoid failures.
2. **Ensures Technical Possibility** – Checks if the system can be built with available technology.
3. **Evaluates Cost-Benefit** – Determines if the system is financially worth developing.
4. **Checks Operational Fit** – Ensures the system will work well with current processes and people.
5. **Supports Decision Making** – Helps management decide whether to **proceed, modify, or cancel** the project.
6. **Saves Time and Resources** – Avoids investing in unfeasible projects.

✅ **In simple words:**

> "Feasibility study **tells if the system is possible, useful, and worth building** before starting development."